Лабораторная работа №12

Программирование в командном процессоре ОС UNIX.
Командные файлы.

true

Цель работы
Выполнение лабораторной работы
2.0 Скрипт с резервным копированием самого себя
Создание директории backup
Получение имени текущего скрипта
Получение полного пути к скрипту
Создание временной метки
Имя архива с временной меткой
Создание архива с помощью tar
Проверка успешности создания
2.1 Обработка произвольного числа аргументов
Способ 1: Цикл for для перебора всех аргументов
Способ 2: Доступ к аргументам > 9
2.2 Аналог команды ls
Если аргумент не передан, используем текущую директорию
Проверка существования директории
Перебор всех файлов в директории
2.3 Подсчёт файлов по расширению
Проверка количества аргументов
Проверка существования директории
Перебор файлов с указанным расширением
Вывод (#вывод)
Контрольные вопросы (#контрольные-вопросы)
Список литературы
СПАСИБО ЗА ВНИМАНИЕ!

#Лабораторная работа №12. Программирование в командном процессоре ОС UNIX. Командные файлы. ## Хайдаров Олим. НБИбд-02-25

Содержание

1. Цель работы
2. Выполнение лабораторной работы - 2.0 Скрипт с резервным копированием самого себя - 2.1 Обработка произвольного числа аргументов - 2.2 Аналог команды ls - 2.3 Подсчёт файлов по расширению
3. Вывод 4. Контрольные вопросы 5. Список литературы

Цель работы

Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать небольшие командные файлы.

Выполнение лабораторной работы

2.0 Скрипт с резервным копированием самого себя

Нам нужно создать скрипт, который:

```
Создаёт резервную копию самого себя
Сохраняет в папку '~/backup'
Архивирует (tar, zip или bzip2)
```

1.

```
fkhayjdarovdk5n18@fedora:~/projects/os-course-project$ cd ~
fkhayjdarovdk5n18@fedora:~$ cd ~/projects
fkhayjdarovdk5n18@fedora:~/projects$ mkdir lab12
fkhayjdarovdk5n18@fedora:~/projects$ cd lab12
fkhayjdarovdk5n18@fedora:~/projects/lab12$ nano task1_backup.sh
fkhayjdarovdk5n18@fedora:~/projects/lab12$ chmod +x task1_backup.sh
fkhayjdarovdk5n18@fedora:~/projects/lab12$
```

Скрипт, создание

Первым делом создаем папку (lab12) и переходим туда командой cdm, далее создаем сам файл (task1_backup.sh) - и вписываем туда код, по которому наша программа будет работать

1.

```bash
#!/bin/bash

# ============================================================
# Лабораторная работа №12, Задание 1
# Скрипт резервного копирования самого себя
# ============================================================

# Создаём директорию backup в домашнем каталоге (если не существует)
mkdir -p ~/backup

# Получаем имя текущего скрипта
SCRIPT_NAME=$(basename "$0")

# Получаем полный путь к скрипту
SCRIPT_PATH="$(readlink -f "$0")"

# Создаём временную метку для уникальности имени архива
TIMESTAMP=$(date +%Y%m%d_%H%M%S)

# Имя архива с временной меткой
BACKUP_NAME="${SCRIPT_NAME}.${TIMESTAMP}.tar.gz"

# Полный путь к архиву
BACKUP_PATH="~/backup/${BACKUP_NAME}"

echo "============================================"
echo "Резервное копирование скрипта: $SCRIPT_NAME"
echo "============================================"
echo "Исходный файл: $SCRIPT_PATH"
echo "Архив: $BACKUP_PATH"
echo ""

# Создаём архив с помощью tar
# tar -czf = tar с сжатием gzip
tar -czf ~/backup/${BACKUP_NAME} -C "$(dirname "$SCRIPT_PATH")" "$(basename "$SCRIPT_PATH")"

# Проверяем, успешно ли создался архив
if [ $? -eq 0 ]; then
    echo "✓ Резервная копия успешно создана!"
    echo "✓ Размер архива: $(du -h ~/backup/${BACKUP_NAME} | cut -f1)"
    echo ""
    echo "Список файлов в backup:"
    ls -lh ~/backup/
else
    echo "✗ Ошибка при создании резервной копии!"
    exit 1
fi
```

Код скрипта

как видно из скрина №1 - мы далее делаем его исполняемым

Код скрипта:

(#!/bin/bash

Создание директории backup

mkdir -p ~/backup

Получение имени текущего скрипта

SCRIPT_NAME=$(basename “$0”)

Получение полного пути к скрипту

SCRIPT_PATH=“$(readlink -f”$0”)”

Создание временной метки

TIMESTAMP=$(date +%Y%m%d_%H%M%S)

Имя архива с временной меткой

BACKUP_NAME=“\({SCRIPT_NAME}.\){TIMESTAMP}.tar.gz”

echo “==========” echo “Резервное копирование скрипта: $SCRIPT_NAME” echo “==========” echo “Исходный файл: \(SCRIPT_PATH" echo "Архив: ~/backup/\){BACKUP_NAME}” echo “”

Создание архива с помощью tar

tar -czf ~/backup/\({BACKUP_NAME} -C "\)(dirname “\(SCRIPT_PATH")" "\)(basename”$SCRIPT_PATH”)”

Проверка успешности создания

if [$? -eq 0]; then echo “✓ Резервная копия успешно создана!” echo “✓ Размер архива: \((du -h ~/backup/\){BACKUP_NAME} | cut -f1)” else echo “✗ Ошибка при создании резервной копии!” exit 1 fi

)

1.

```
ofkhayjdarovdk5n18@fedora:~/projects/lab12$ ./task1_backup.sh
==========================================
Резервное копирование скрипта: task1_backup.sh
==========================================
Исходный файл: /home/ofkhayjdarovdk5n18/projects/lab12/task1_backup.sh
Архив: ~/backup/task1_backup.sh.20260502_185249.tar.gz

✓ Резервная копия успешно создана!
✓ Размер архива: 4.0K

Список файлов в backup:
total 748K
-rw-r--r--. 1 ofkhayjdarovdk5n18 ofkhayjdarovdk5n18 742K May  2 17:50 bash.20260
502_174928.tar.gz
-rw-r--r--. 1 ofkhayjdarovdk5n18 ofkhayjdarovdk5n18  933 May  2 18:52 task1_back
up.sh.20260502_185249.tar.gz
ofkhayjdarovdk5n18@fedora:~/projects/lab12$
```

Запуск скрипта

мы запустили скрипт и он успешно заработал

2.1 Обработка произвольного числа аргументов

Нам так же, как и в задании №1 нужно создать файл (tasks2_args.sh) и далее написать код для обработки произвольного числа аргументов (больше 10)

1.

```
ofkhayjdarovdk5n18@fedora:~/projects/os-course-project$ cd lab12
bash: cd: lab12: No such file or directory
ofkhayjdarovdk5n18@fedora:~/projects/os-course-project$ cd ~/projects
ofkhayjdarovdk5n18@fedora:~/projects$ cd lab12
ofkhayjdarovdk5n18@fedora:~/projects/lab12$ nano tasks2_args.sh
ofkhayjdarovdk5n18@fedora:~/projects/lab12$ chmod +x task2_args.sh
chmod: cannot access 'task2_args.sh': No such file or directory
ofkhayjdarovdk5n18@fedora:~/projects/lab12$
ofkhayjdarovdk5n18@fedora:~/projects/lab12$ chmod +x tasks2_args.sh
ofkhayjdarovdk5n18@fedora:~/projects/lab12$
```

Создание файла

2.

```
GNU nano 8.5                                                    tasks2_args.sh
#!/bin/bash

# ============================================================
# Лабораторная работа №12, Задание 2
# Обработка произвольного числа аргументов (в т.ч. > 10)
# ============================================================

echo "==================================================="
echo "Всего передано аргументов: $#"
echo "==================================================="

# Способ 1: Цикл for (самый надежный способ перебрать ВСЕ аргументы)
echo "Список всех аргументов:"
count=1
for arg in "$@"
do
    echo "  Аргумент №$count: $arg"
    count=$((count + 1))
done

echo ""
echo "==================================================="

# Способ 2: Явный доступ к аргументам > 9
# В Bash к аргументам начиная с 10-го нужно обращаться через ${10}, ${11} и т.д.
if [ $# -ge 10 ]; then
    echo "Проверка доступа к аргументам > 9:"
    echo "  10-й аргумент: ${10}"
    if [ $# -ge 11 ]; then
        echo "  11-й аргумент: ${11}"
    fi
fi
```

Код скрипта

(Далее, как показано в скриншоте №1 - мы делаем этот файл исполняемым - командой chmod)

Полный код скрипта:

(#!/bin/bash

echo “==” echo “Всего передано аргументов: $#” echo “==”

Способ 1: Цикл for для перебора всех аргументов

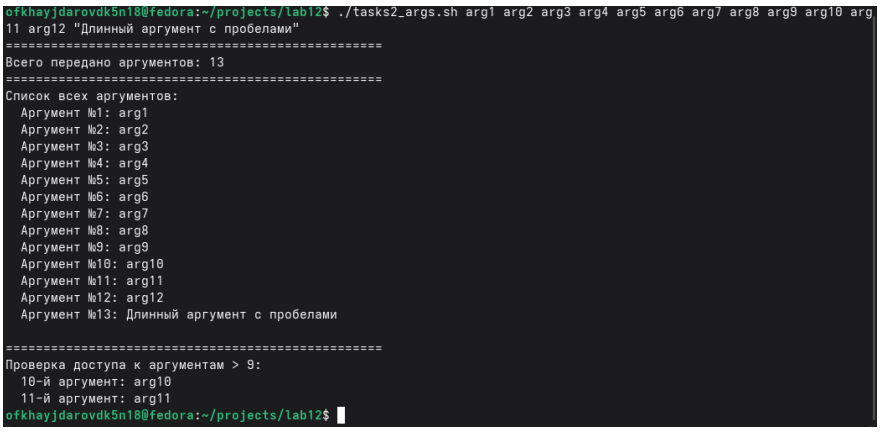
echo “Список всех аргументов:” count=1 for arg in “\(@" do echo " Аргумент №\)count: \(arg" count=\)((count + 1)) done

echo “” echo “==”

Способ 2: Доступ к аргументам > 9

if [$# -ge 10]; then echo “Проверка доступа к аргументам > 9:” echo ” 10-й аргумент: ${10}” if [$# -ge 11]; then echo ” 11-й аргумент: ${11}” fi fi

)

1. 

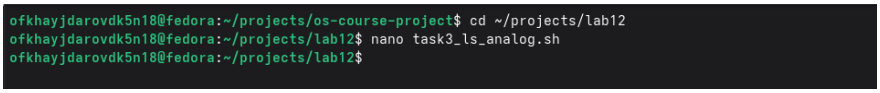
   Запуск файла

Проверяем работу файла, запуская его командой ’ ./tasks2_args.sh arg1 arg2 arg3 arg4 arg5 arg6 arg7 arg8 arg9 arg10 arg11 arg12 “Длинный аргумент с пробелами” ’

В конце получем результат - команда успешно и корректно заработала

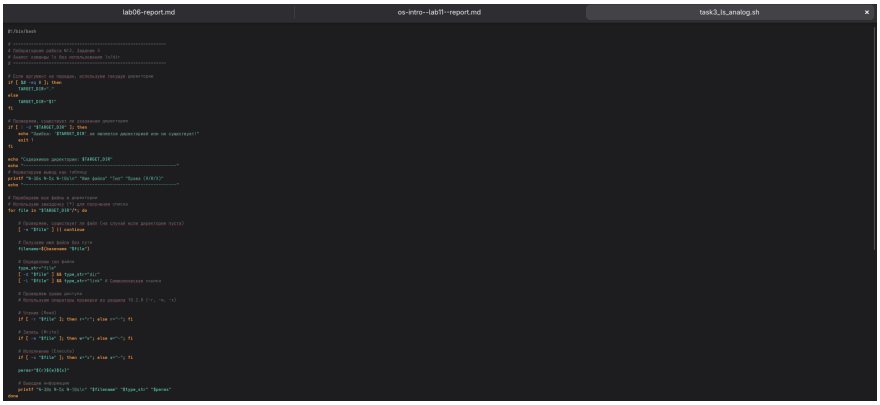
2.2 Аналог команды ls

В этом задании нужно создайть скрипт, который выводит информацию о файлах (без ls и dir):

1. 

Создание файла

- вводим chmod – делая его исполняемым (чтобы он мог быть оперируемым в наших итерациях с ним) – после того, как написали скрипт в файле

1. 

Код скрипта

Полный код скрипта (листинг): (#!/bin/bash

Если аргумент не передан, используем текущую директорию

if [$# -eq 0]; then TARGET_DIR=“.” else TARGET_DIR=“$1” fi

Проверка существования директории

if [! -d “\(TARGET_DIR"]; then echo "Ошибка: '\)TARGET_DIR’ не является директорией!” exit 1 fi

echo “Содержимое директории: $TARGET_DIR” echo “——————————————————————————” printf “%-30s %-5s %-10s” “Имя файла” “Тип” “Права (R/W/X)” echo “——————————————————————————”

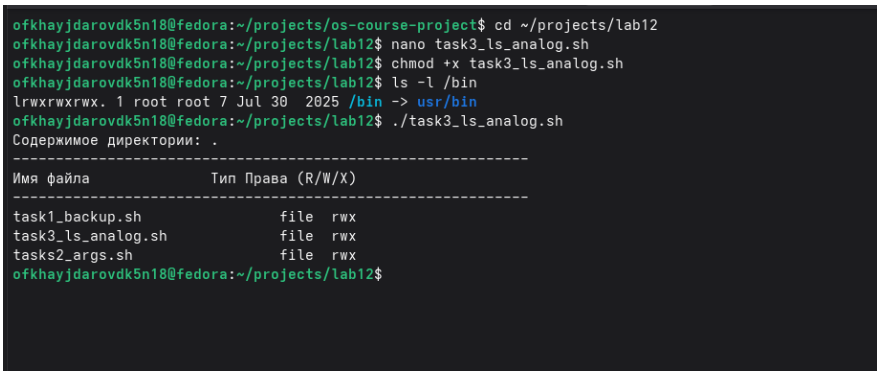
Перебор всех файлов в директории

for file in “\(TARGET_DIR"/*; do [-e "\)file”] || continue

```
filename=$(basename "$file")

# Определение типа файла
type_str="file"
[ -d "$file" ] && type_str="dir"
[ -L "$file" ] && type_str="link"

# Проверка прав доступа
if [ -r "$file" ]; then r="r"; else r="-"; fi
if [ -w "$file" ]; then w="w"; else w="-"; fi
if [ -x "$file" ]; then x="x"; else x="-"; fi

perms="${r}${w}${x}"

printf "%-30s %-5s %-10s\n" "$filename" "$type_str" "$perms"
```

done

)

1. 

   Запуск файла

Запускаем код файла (сделали на скриншоте 2) командой:

’ ./task3_ls_analog.sh ’

Нам выводит сообщение об успешной работе кода файла

2.3 Подсчёт файлов по расширению

4-е задание нам говорит о создании скрипта, который считает файлы определенного расширения (количественно).

1.

```
ofkhayjdarovdk5n18@fedora:~/projects/os-course-project$ cd ~/projects/lab12
ofkhayjdarovdk5n18@fedora:~/projects/lab12$ nano task4_count.sh
ofkhayjdarovdk5n18@fedora:~/projects/lab12$ chmod +x task4_count.sh
ofkhayjdarovdk5n18@fedora:~/projects/lab12$
```

Создание файла

- (вводим chmod – делая его исполняемым (чтобы он мог быть оперируемым в наших итерациях с ним) – после того, как мы написали скрипт в файле)

1.

```bash
#!/bin/bash

# ============================================================
# Лабораторная работа №12, Задание 4
# Подсчет файлов по расширению в указанной директории
# ============================================================

# Проверка количества аргументов (нужно 2: расширение и путь)
if [ $# -lt 2 ]; then
    echo "Ошибка: Недостаточно аргументов."
    echo "Использование: $0 <расширение> <путь_к_директории>"
    echo "Пример: $0 .sh /home/user/projects"
    exit 1
fi

EXTENSION="$1"
TARGET_DIR="$2"

# Проверка, существует ли указанная директория
if [ ! -d "$TARGET_DIR" ]; then
    echo "Ошибка: Директория '$TARGET_DIR' не найдена."
    exit 1
fi

echo "Поиск файлов с расширением '$EXTENSION' в директории: $TARGET_DIR"
echo "------------------------------------------------------------"

COUNT=0

# Перебираем все файлы, совпадающие с шаблоном *расширение
# Используем кавычки, чтобы пути с пробелами обрабатывались верно
for file in "$TARGET_DIR"/*"$EXTENSION"; do

    # Проверка: существует ли файл (на случай если маска ничего не нашла)
    # и является ли он обычным файлом (а не папкой)
    if [ -f "$file" ]; then
        COUNT=$((COUNT + 1))
        # Можно раскомментировать строку ниже, чтобы видеть список найденных файлов
        # echo "Найден: $(basename "$file")"
    fi
done

echo "------------------------------------------------------------"
echo "Всего найдено файлов: $COUNT"
```

Код скрипта

Полный код скрипта (листинг):

(#!/bin/bash

Проверка количества аргументов

if [$# -lt 2]; then echo “Ошибка: Недостаточно аргументов.” echo “Использование: $0 <расширение> <путь_к_директории>” exit 1 fi

EXTENSION=“$1” TARGET_DIR=“$2”

Проверка существования директории

if [! -d “\(TARGET_DIR"]; then echo "Ошибка: Директория '\)TARGET_DIR’ не найдена.” exit 1 fi

echo “Поиск файлов с расширением ‘$EXTENSION’ в директории: $TARGET_DIR” echo “————————————————————————————”

COUNT=0

Перебор файлов с указанным расширением

for file in “\(TARGET_DIR"/*"\)EXTENSION”; do if [-f “\(file"]; then COUNT=\)((COUNT + 1)) fi done

echo “————————————————————————————” echo “Всего найдено файлов: $COUNT”

)

1.

```
ofkhayjdarovdk5n18@fedora:~/projects/os-course-project$ cd ~/projects/lab12
ofkhayjdarovdk5n18@fedora:~/projects/lab12$ nano task4_count.sh
ofkhayjdarovdk5n18@fedora:~/projects/lab12$ chmod +x task4_count.sh
ofkhayjdarovdk5n18@fedora:~/projects/lab12$ ./task4_count.sh
Ошибка: Недостаточно аргументов.
Использование: ./task4_count.sh <расширение> <путь_к_директории>
Пример: ./task4_count.sh .sh /home/user/projects
ofkhayjdarovdk5n18@fedora:~/projects/lab12$ ./task4_count.sh .sh .
Поиск файлов с расширением '.sh' в директории: .
------------------------------------------------------------
------------------------------------------------------------
Всего найдено файлов: 4
ofkhayjdarovdk5n18@fedora:~/projects/lab12$
```

Запуск файла

Далее запускаем файл командой:

'

```
./task4_count.sh .sh .
```

,

В конце получаем результат - успешная и корректная работа скрипта файла

Вывод (#вывод)

Выполняя лабораторную работу №12 я изучил основы программирования в оболочке ОС UNIX/Linux и научился писать базовые скрипты/командные файлы для выполнения тех или иных задач.

Контрольные вопросы (#контрольные-вопросы)

1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются?

Командная оболочка (интерпретатор команд, shell) — это программа, позволяющая пользователю взаимодействовать с операционной системой, запуская команды и интерпретируя скрипты.

Примеры:

’ sh (Bourne shell): Стандартная оболочка UNIX с базовым набором функций. csh (C-shell): Надстройка над sh, использующая C-подобный синтаксис и поддерживающая историю команд. ksh (Korn shell): Напоминает C-оболочку, но совместима с синтаксисом sh. BASH (Bourne Again Shell): Разработка FSF, совмещает свойства C-оболочки и Korn shell, является стандартом для Linux. Отличия: Они различаются синтаксисом команд,

способами управления историей и возможностями программирования (хотя большинство базовых команд совпадает). ’

1. Что такое POSIX?

POSIX (Portable Operating System Interface for Computer Environments) — это набор стандартов, разработанных комитетом IEEE для описания интерфейсов взаимодействия ОС и приложений. Цель — обеспечить совместимость различных UNIX/Linux-подобных систем и переносимость прикладных программ на уровне исходного кода. POSIX-совместимые оболочки разработаны на базе Korn shell (ksh).

1. Как определяются переменные и массивы в языке программирования bash?

   Переменные: Определяются присваиванием значения строки символов без пробелов вокруг знака равенства: имя=значение (например, mark=/usr/andy/bin). Для доступа к значению используется знак доллара: $имя или ${имя}. Массивы: Создаются с помощью команды set -A имя_массива val1 val2 .... Индексация начинается с нуля. Добавить элемент можно так: имя_массива[49]=Alaska.

2. Каково назначение операторов let и read?

’ let: Используется для арифметических вычислений. Он воспринимает следующие аргументы как выражение, подлежащее вычислению (например, let sum=x+7). Поддерживает сложение, вычитание, побитовые операции и т.д. read: Позволяет читать значения переменных со стандартного ввода (с клавиатуры). Например: read mon day считает два слова и запишет их в переменные mon и day. ’

1. Какие арифметические операции можно применять в языке программирования bash?

В bash поддерживаются стандартные математические и логические операции (см. Таблицу 10.1 в методичке):

’ Арифметические: + (сложение), - (вычитание), * (умножение), / (целочисленное деление), % (остаток от деления). Логические и побитовые: && (AND), || (OR), ! (NOT), &, |, ^. Сравнения: <, >, <=, >=, ==, !=. Сдвиги: << (влево), >> (вправо). ’

1. Что означает операция (())?

Двойные круглые скобки используются для записи сложных арифметических выражений и условий без необходимости писать ключевое слово let. Внутри скобок ноль воспринимается как «ложь», а любое другое значение — как «истина». Это удобно для использования в условиях циклов, например: while ((x > 0)).

1. Какие стандартные имена переменных Вам известны?

   PATH: Список каталогов для поиска команд. HOME: Путь к домашнему каталогу пользователя. IFS: Разделители в командной строке (пробел, табуляция, перевод строки). MAIL: Путь к файлу входящей почты. TERM: Тип используемого терминала. LOGNAME: Имя пользователя. PS1, PS2: Строки промптеров (приглашений командной строки).

2. Что такое метасимволы?

Метасимволы — это символы, имеющие для командного процессора специальный смысл (управляющие символы).

Примеры:

’ * — соответствует любой строке (включая пустую). ? — соответствует любому одиночному символу. [c1-c2] — соответствует любому символу из диапазона. <, >, | — используются для перенаправления потоков ввода-вывода и конвейеров. ’

1. Как экранировать метасимволы?

Экранирование — это снятие специального смысла с метасимвола:

```
Использованием обратного слэша \ перед символом (например, echo \* выведет просто звездочку).
Заключением группы метасимволов в одинарные кавычки '...' (экранирует всё внутри).
```

```
Заключением в двойные кавычки "..." (экранирует большинство
метасимволов, но сохраняет спецсмысл $, \, " и `).
```

1. Как создавать и запускать командные файлы?

   Создать текстовый файл с набором команд. Запускать файл командой bash имя_файла [аргументы]. Чтобы запускать файл просто по имени, нужно изменить права доступа, добавив бит исполнения: chmod +x имя_файла.

2. Как определяются функции в языке программирования bash?

Функции определяются с помощью ключевого слова function или круглых скобок:

function fname { команды } # или fname() { команды }

Для удаления функции используется команда unset -f fname.

1. Каким образом можно выяснить, является файл каталогом или обычным файлом?

С помощью команды test или квадратных скобок [...]:

```
test -f file (или [ -f file ]) — истина, если файл существует
(обычный файл).
test -d file (или [ -d file ]) — истина, если файл является
каталогом.
Также можно проверить права: test -r file (чтение), test -w file
(запись), test -e file (исполнение).
```

1. Каково назначение команд set, typeset и unset?

   set: Выводит список всех переменных и функций; используется для создания массивов (set -A). typeset: Позволяет задавать атрибуты переменных. Например, typeset -i объявляет переменную целочисленной, typeset -f перечисляет функции, typeset -ft включает трассировку. unset: Изъятие переменной или функции из памяти (например, unset -f для удаления функции).

2. Как передаются параметры в командные файлы?

Параметры передаются при вызове скрипта через пробел (например, ./script arg1 arg2). Внутри скрипта они доступны как позиционные переменные: $0 (имя файла), $1 (первый аргумент), $2 (второй) и т.д. Для доступа к аргументам с номером больше 9

используются фигурные скобки: ${10}, ${11}. Специальная переменная $# хранит общее количество переданных параметров.

1. Назовите специальные переменные языка bash и их назначение.

’ $# — количество аргументов. $* и $@ — все аргументы как единое целое (разница в обработке пробелов). $? — код завершения последней команды (0 — успех). $$ — уникальный идентификатор процесса (PID) текущего shell. $! — PID последнего процесса, запущенного в фоновом режиме. $- — текущие флаги командного процессора. ${#name} — длина строки в переменной name. ’

Список литературы

Список литературы:

Основные источники

Кулябов Д. С., Королькова А. В., Микеладзе Ш. А., Сергеев А. В. Операционные системы: учебное пособие. — М.: РУДН, 2024. — С. 82–96.
GNU Bash Manual. Free Software Foundation, 2026. — URL: https://www.gnu.org/software/bash/manual/ (дата обращения: 02.05.2026).
Advanced Bash-Scripting Guide. Mendel Cooper, 2024. — URL: https://tldp.org/LDP/abs/html/ (дата обращения: 02.05.2026).

Дополнительная литература

Левин М. Г. Теоретические основы работы в ОС UNIX. — М.: ДМК Пресс, 2020. — 384 с.
Неммет А. Bash: карманный справочник. — 2-е изд. — СПб.: Символ-Плюс, 2019. — 176 с.
Рошан П., Хоффман Д. Linux. Командная строка. Полное руководство. — М.: Эксмо, 2021. — 512 с.
Собелл М. Г. Практическое руководство по Linux. Команды, редакторы и прикладное программное обеспечение. — 4-е изд. — М.: Вильямс, 2022. — 1024 с.

Электронные ресурсы

The Linux Documentation Project (TLDP). Bash scripting tutorials. — URL: https://tldp.org/HOWTO/Bash-Prog-Intro-HOWTO.html (дата обращения: 02.05.2026).
ShellCheck. Static analysis tool for shell scripts. — URL: https://www.shellcheck.net/ (дата обращения: 02.05.2026).
ExplainShell. Interactive shell command explanation. — URL: https://explainshell.com/ (дата обращения: 02.05.2026).
Hugo Documentation. Static site generator. — URL: https://gohugo.io/documentation/ (дата обращения: 02.05.2026).
GitHub Pages Documentation. — URL: https://docs.github.com/en/pages (дата обращения: 02.05.2026).

Стандарты

IEEE Std 1003.1-2017. Standard for Information Technology — Portable Operating System Interface (POSIX®). — IEEE Computer Society, 2018.
POSIX.1-2008. The Open Group Base Specifications Issue 7. — The Open Group, 2018. — URL: https://pubs.opengroup.org/onlinepubs/9699919799/ (дата обращения: 02.05.2026).

СПАСИБО ЗА ВНИМАНИЕ!